

BANCO DE DESARROLLO PRODUCTIVO

BDP S.A.M.

MANUAL DE INSTALACION

Proyecto: BDP — Sistema de Análisis de Microcrédito (S.A.M.)
Versión 1.0 — Mayo 2026

Proyecto	BDP — Sistema de Análisis de Microcrédito (S.A.M.)
Versión	1.0
Fecha	Mayo 2026
Clasificación	Uso interno — Confidencial

1. BIENVENIDO AL EQUIPO

Este manual te guiará paso a paso para configurar tu computadora, descargar el proyecto, entender la arquitectura y empezar a desarrollar en el sistema BDP S.A.M.

No necesitas experiencia previa con este stack. Todo está explicado desde cero.

2. HERRAMIENTAS QUE DEBES INSTALAR

2.1 Java JDK 17

¿Qué es? El lenguaje de programación del backend.

Pasos:

1. Ve a: <https://adoptium.net/temurin/releases/?version=17>
2. Descarga el instalador para Windows (.msi)
3. Ejecuta el instalador (Siguiente → Siguiente → Finalizar)
4. Abre una terminal (CMD o PowerShell) y verifica:

```
java -version
```

Debe mostrar: `openjdk version "17.0.x"`

2.2 Node.js 20

¿Qué es? El entorno para ejecutar el frontend React.

Pasos:

5. Ve a: <https://nodejs.org/>
6. Descarga la versión LTS (20.x)
7. Ejecuta el instalador (Siguiente → Siguiente → Finalizar)
8. Verifica en la terminal:

```
node --version  
npm --version
```

Debe mostrar: `v20.x.x` y `10.x.x`

2.3 Git

¿Qué es? Sistema de control de versiones para descargar y compartir código.

Pasos:

9. Ve a: <https://git-scm.com/download/win>

10. Descarga el instalador (64-bit)
11. Ejecuta el instalador. Importante: en la pantalla "Choosing the default editor", selecciona "Use Visual Studio Code as Git's default editor"
12. Verifica en la terminal:

```
git --version
```

Debe mostrar: git version 2.x.x

2.4 Visual Studio Code (VSCode)

¿Qué es? El editor de código donde trabajaremos.

Pasos:

13. Ve a: <https://code.visualstudio.com/>
14. Descarga e instala
15. Ábrelo una vez instalado

2.5 Docker Desktop

¿Qué es? Programa que crea "contenedores" para simular servidores en tu PC.

Pasos:

16. Ve a: <https://www.docker.com/products/docker-desktop/>
17. Descarga e instala
18. Reinicia tu computadora después de instalar
19. Abre Docker Desktop (ícono de ballena)
20. Acepta los términos de uso
21. Espera a que el ícono en la bandeja del sistema esté verde (Engine Running)

2.6 GitHub Desktop

¿Qué es? Programa para manejar Git con interfaz gráfica (más fácil que comandos).

Pasos:

22. Ve a: <https://desktop.github.com/>
23. Descarga e instala
24. Inicia sesión con tu cuenta de GitHub
25. Si no tienes cuenta, créala en: <https://github.com/signup>

3. EXTENSIONES DE VSCode (OBLIGATORIAS)

Abre VSCode y ve al panel de extensiones (Ctrl+Shift+X). Instala estas:

Extensión	Para qué sirve
-----------	----------------

Extension Pack for Java	Soporte para Java y Spring Boot
Spring Boot Extension Pack	Ejecutar backend desde VSCode
ES7+ React Snippets	Atajos para código React
Prettier - Code formatter	Formateo automático de código
Docker	Ver contenedores desde VSCode

¿Cómo instalar? Busca el nombre, haz click en "Install" y espera a que termine.

4. DESCARGAR EL PROYECTO

4.1 Clonar el repositorio

1. Abre GitHub Desktop
2. Ve a: File → Clone Repository
3. Selecciona la pestaña URL
4. Pega esta URL:

```
https://github.com/diegoMB3/CONSULTORIA-PARA-LA-PROVISION-DE-SOFTWARE-PARA-PYME-Y-BY.git
```

5. En "Local Path", elige: C:\BDP-SAM-Project
6. Haz click en Clone

Resultado: Tendrás el proyecto en C:\BDP-SAM-Project\

4.2 Abrir en VSCode

7. Abre VSCode
8. Ve a: File → Open Folder
9. Selecciona: C:\BDP-SAM-Project
10. Haz click en "Yes, I trust the authors"

Ahora verás todas las carpetas del proyecto en el panel izquierdo.

5. ESTRUCTURA DEL PROYECTO

```
C:\BDP-SAM-Project\  
├── backend/ ← Código del servidor (Java + Spring Boot)  
│   ├── pom.xml ← Dependencias del backend  
│   └── src/main/java/bo/gob/bdp/sam/  
│       ├── core/ ← Lógica de negocio (Arquitectura Hexagonal)  
│       │   ├── application/command/ ← Comandos CQRS  
│       │   └── domain/  
│       └── aggregate/ ← Agregados (reglas de negocio)
```

└─ event/	← Eventos inmutables
└─ adapters/in/web/	← Controladores REST (API)
└─ web-dashboard/	← Código de la página web (React + TypeScript)
└─ package.json	← Dependencias del frontend
└─ src/pages/	← Pantallas de la aplicación
└─ MenuPrincipal.tsx	
└─ RegistroCliente.tsx	
└─ ChecklistDocumentos.tsx	
└─ VolteoBalances.tsx	
└─ EvaluacionCliente.tsx	
└─ infra/	← Configuración de servidores (Docker)
└─ docker-compose.yml	← Servicios: PostgreSQL, Axon, WireMock
└─ mobile/	← App Android (Futuro)

6. CÓMO EJECUTAR EL PROYECTO

6.1 Iniciar los servidores Docker

¿Qué son? Base de datos, bus de eventos y simulador del banco.

11. Abre una terminal (Cmd o PowerShell)
12. Ejecuta:

```
cd C:\BDP-SAM-Project\infra
docker-compose up -d
```

13. Espera a que descargue las imágenes (solo la primera vez)
14. Verifica que estén corriendo:

```
docker ps
```

Debes ver 3 contenedores: bdp_database, bdp_axon_server, bdp_core_mock

6.2 Iniciar el Backend

15. Abre VSCode
16. En el panel izquierdo, busca: backend/src/main/java/bo/gob/bdp/sam/BackendApplication.java
17. Haz click derecho → Run Java
18. O abre una terminal y ejecuta:

```
cd C:\BDP-SAM-Project\backend
.\mvnw.cmd spring-boot:run
```

Espera a que aparezca: Started BackendApplication in X seconds

El backend está corriendo en: <http://localhost:8080>

6.3 Iniciar el Frontend

- 19. Abre otra terminal (sin cerrar la del backend)
- 20. Ejecuta:

```
cd C:\BDP-SAM-Project\web-dashboard
npm install          ← Solo la primera vez
npm run dev

Espera a que aparezca: Local: http://localhost:5173/
```

El frontend está corriendo en: <http://localhost:5173>

6.4 URLs de Prueba

URL	Módulo	¿Qué hace?
http://localhost:5173/	Menú Principal	Muestra 3 modelos de evaluación (Agrícola, Pecuario, Industrial)
http://localhost:5173/registro-cliente	Registro de Cliente	Formulario para crear un nuevo cliente
http://localhost:5173/checklist/1234567	Checklist de Documentos	Marcar documentos recibidos
http://localhost:5173/volteo-balances/1234567	Volteo de Balances	Ingresar datos financieros con validación

7. ¿QUÉ HEMOS CONSTRUIDO HASTA AHORA?

7.1 Requisitos Implementados (4 de 14)

Requisito	Descripción Simple	¿Dónde está?
RF-AUTH-01	Menú principal con modelos de evaluación	MenuPrincipal.tsx + ModeloController.java
RF-02.1	Formulario para registrar un cliente nuevo	RegistroCliente.tsx + ClienteAggregate.java
RF-02.2	Checklist de documentos (bloquea si faltan papeles)	ChecklistDocumentos.tsx + DocumentoAggregate.java
RF-03.1	Ingreso de datos financieros con validación de ±10 Bs	VolteoBalances.tsx + BalanceAggregate.java

7.2 Lo que falta por construir

Prioridad	Requisito	Descripción Simple
-----------	-----------	--------------------

1	RF-03.2	Indicadores Financieros (liquidez, solvencia)
2	RF-03.3	Simulador de Pagos (cronograma de cuotas)
3	RF-04.1	Evaluación Agrícola (costos por hectárea)
4	RF-05.1	Evaluación Pecuaria (proyección de ganado)
5	RF-06.1	Evaluación Producción (costos industriales)
6-10	RF-07.1 a RF-12.1	Flujo de aprobación, PDFs, auditoría, app Android

8. CONCEPTOS CLAVE QUE DEBES ENTENDER

8.1 ¿Qué es Arquitectura Hexagonal?

Imagina un corazón (la lógica de negocio) y conectores (la API, la base de datos).

- core/ → El corazón. Aquí están las reglas de negocio. NO depende de nada externo.
- adapters/ → Los conectores. Conectan el corazón con el mundo exterior (HTTP, base de datos).

Beneficio: Si el banco cambia de Oracle a PostgreSQL, solo cambias un adaptador, no tocas la lógica.

8.2 ¿Qué es Event Sourcing?

En lugar de guardar solo el estado final (ej: "Cliente Juan, teléfono 7777"), guardamos cada cambio como un evento:

- ClienteRegistrado → Juan fue creado
- TelefonoActualizado → Juan cambió su teléfono a 7777
- EmailActualizado → Juan cambió su email

Beneficio: Sabemos quién hizo qué, cuándo y cómo. Exigencia de la ASFI para auditoría.

8.3 ¿Qué es CQRS?

Separamos las operaciones de escritura (crear, modificar) de las de lectura (consultar, reportes).

- Command Side → *Command.java → *Aggregate.java (Escritura)
- Query Side → *Controller.java (Lectura, actualmente en memoria)

Beneficio: Los reportes no hacen lento el sistema cuando alguien está registrando un cliente.

8.4 ¿Cómo se conecta todo?

Navegador (React) → Backend (Spring Boot) → Axon Server (Eventos)

```
└─ PostgreSQL (Datos)
└─ WireMock (Simula el banco)
```

21. El usuario llena un formulario en React
22. React envía los datos al Backend (HTTP)
23. El Backend valida las reglas de negocio
24. El Backend guarda un evento inmutable en Axon Server
25. El Backend responde al frontend
26. El frontend muestra el resultado al usuario

9. COMANDOS ÚTILES PARA EL DÍA A DÍA

Docker

```
# Iniciar servidores
cd C:\BDP-SAM-Project\infra
docker-compose up -d

# Ver si están corriendo
docker ps

# Detener servidores
docker-compose down

# Ver logs de Axon
docker logs bdp_axon_server -f
```

Backend

```
# Compilar y ejecutar
cd C:\BDP-SAM-Project\backend
.\mvnw.cmd clean spring-boot:run

# Solo compilar (para verificar que no hay errores)
.\mvnw.cmd clean compile
```

Frontend

```
# Instalar dependencias (primera vez o cuando se agregan librerías)
cd C:\BDP-SAM-Project\web-dashboard
npm install

# Ejecutar en modo desarrollo
npm run dev

# Compilar para producción
npm run build
```

Git (GitHub Desktop)

```
# Ver cambios pendientes
git status

# Agregar todos los cambios
git add .

# Crear un commit
git commit -m "feat: descripción de lo que hiciste"

# Subir a GitHub
git push origin main

# Bajar cambios de otros
git pull origin main
```

10. CONSEJOS PARA EMPEZAR

Primer día

27. Instala todas las herramientas (Sección 2)
28. Clona el proyecto (Sección 4)
29. Ejecuta el backend y frontend (Sección 6)
30. Navega por las 4 URLs de prueba

Primera semana

31. Lee los archivos de la carpeta core/domain/aggregate/ para entender la lógica
32. Lee los archivos de la carpeta web-dashboard/src/pages/ para entender las pantallas
33. Practica haciendo un cambio pequeño (ej: cambiar un texto en MenuPrincipal.tsx)
34. Haz commit y push de tu cambio

Cuando tengas dudas

- Revisa este manual (Sección 8: Conceptos Clave)
- Pregunta al equipo
- Revisa los archivos de ejemplo que ya están hechos

11. GLOSARIO RÁPIDO

Término	Significado Simple
Backend	El servidor. Procesa datos, guarda en base de datos.
Frontend	La página web. Lo que ve y usa el cliente.
API REST	La forma en que el frontend habla con el backend.
Docker	Crea 'mini computadoras virtuales' para servidores.
Git	Guarda el historial de cambios del código.
Commit	Un 'save point' del código.

Push	Subir tus cambios a GitHub.
Pull	Bajar cambios de otros a tu PC.
RF	Requerimiento Funcional (lo que el sistema debe hacer).
RNF	Requerimiento No Funcional (cómo debe comportarse).

Documento elaborado para el proyecto BDP S.A.M. | Mayo 2026 | Versión 1.0